

# Criptografia em Java

## 1 Introdução

Para a realização deste trabalho é necessário ter o Java Development Kit (JDK), versão 8 ou posterior, instalado no seu computador. Pode usar o JDK do OpenJava (em Linux, instalar com o comando `apt-get`) ou o da Oracle, que pode obter na página oficial de distribuição da versão SE ( Standard Edition ) do Java<sup>1</sup>. Poderá também instalar um IDE da sua preferência, como o NetBeans<sup>2</sup>, por exemplo.

Na realização deste trabalho é necessário visualizar/alterar o conteúdo de ficheiros em binário. Para esse efeito, e se estiver em ambiente Linux, pode instalar a aplicação GHex , disponível no repositório de aplicações do Ubuntu.

## 2 Criptografia de chave simétrica

As seguintes classes que aqui se apresentam são fundamentais para o desenvolvimento de aplicações envolvendo criptografia. Pode encontrar informação detalhada sobre estas e outras classes no Security Developers Guide, no site da Oracle<sup>3</sup>.

### KeyGenerator

A classe `KeyGenerator`, do pacote `javax.crypto`, implementa um gerador de chaves simétricas. Alguns métodos importantes desta classe são: `getInstance` e `generateKey`. Use o JavaDoc para obter informação sobre a classe e os métodos indicados.

### SecretKey

A classe `SecretKey`, do pacote `javax.crypto`, implementa uma chave simétrica. Um método importante desta classe é o `getEncoded`. Use o JavaDoc para obter informação sobre a classe e o método indicado.

### SecretKeySpec

A classe `SecretKeySpec`, do pacote `javax.crypto.spec`, permite criar uma chave simétrica a partir de um array de bytes (`byte []`). Use o JavaDoc para obter informação sobre ela e sobre os seus métodos.

### Cipher

A classe `Cipher`, do pacote `javax.crypto`, permite realizar operações de cifra e decifra. Alguns métodos importantes desta classe são: `getInstance` , `init` e `doFinal`. Use o JavaDoc para obter informação sobre a classe e os métodos indicados.

### 2.1 Cifra simétrica por blocos

Nesta secção pretende-se explorar a cifra simétrica por blocos usando o algoritmo AES no modo de cifra ECB. Na parte final estende-se a outros algoritmos, mas sempre utilizando o modo de cifra ECB.

---

1 <https://www.oracle.com/technetwork/java/javase/downloads/index.html>

2 <https://netbeans.apache.org/>

3 <https://docs.oracle.com/en/java/javase/11/security/index.html>

**NOTA:** Tenha em atenção que todas as operações de leitura/escrita para ficheiros dos dados a cifrar, dos criptogramas produzidos, das chaves e dos vetores iniciais devem ser sempre realizadas em modo binário.

### 2.1.1 Geração de uma chave simétrica

Desenvolva um pequeno programa para gerar uma chave simétrica para o algoritmo AES e guardá-la num ficheiro cujo nome é fornecido como parâmetro de entrada.

Nota: se quiser imprimir a chave gerada no ecrã, tenha em atenção que ela não é uma String de caracteres de texto mas sim um array de bytes, pelo que não conseguirá ver o seu conteúdo de forma adequada. Para poder visualizar a chave deve primeiro transformá-la numa String, usando uma de duas codificações: Formato Base 64 (Classe Base64) ou formato Hexadecimal (classe HexFormat)

Como sabe, a aleatoriedade das chaves é importante em criptografia, pelo que se deve gerar bons geradores de números aleatórios. Por omissão o Java inclui um gerador de números aleatórios (SecureRandom) que pode usar em conjunto com o seu gerador de chaves. Altere o seu programa para que use um gerador de números aleatórios na geração de chaves.

### 2.1.2 Cifra de um ficheiro usando criptografia simétrica

Para realizar uma operação de cifra é necessário indicar o algoritmo de cifra, o modo de cifra e o algoritmo de padding a usar. O Java designa o conjunto destes 3 elementos como transformação e a sua especificação é feita numa String com a forma "Algoritmo/ModoCifra/Padding", como no seguinte exemplo "AES/ECB/NoPadding".

Crie um programa para cifrar o conteúdo de um ficheiro usando o algoritmo de cifra AES. O programa deve ter os seguintes parâmetros de entrada: (1) o nome do ficheiro com o conteúdo em claro para cifrar; (2) o nome pretendido para o ficheiro onde guardar o criptograma (conteúdo cifrado) produzido; (3) o nome do ficheiro com a chave simétrica a usar.

Nota : Na instanciação do objecto Cypher, use como transformação a String "AES/CBC/PKCS5Padding".

### 2.1.3 Decifra de um ficheiro usando criptografia simétrica

Desenvolva um novo programa para decifrar o conteúdo de um ficheiro cifrado utilizando o algoritmo AES. O programa deve ter os seguintes parâmetros de entrada: (1) o nome do ficheiro com o conteúdo para decifrar, (2) o nome pretendido para o ficheiro onde guardar o conteúdo decifrado e (3) o nome do ficheiro com a chave a utilizar na operação de decifra.

### 2.1.4 Cifra e decifra de um ficheiro usando um algoritmo a especificar pelo utilizador

Altere os programas anteriores para que a transformação seja passada como um parâmetro de entrada. O objetivos desta alteração é permitir a utilização do seu programa com várias transformações, de acordo com o interesse do utilizador.

Tenha em atenção que alguns algoritmos permitem vários tamanhos de chave, como é o caso do AES que permite tamanhos de 128, 192 e 256 bits, pelo que também deve permitir a indicação do tamanho da chave como parâmetro de entrada e ter o cuidado de gerar a chave de acordo com o tamanho indicado.

### 2.1.5 Alinhamento com excipiente (*padding*)

Como sabe, um algoritmo de cifra por blocos, apenas cifra conteúdos de dimensão fixa e igual ao tamanho do seu bloco. No entanto, dificilmente um conteúdo que se pretende cifrar tem uma dimensão múltipla do tamanho do bloco do algoritmo que se pretende usar. Para resolver este problema, faz-se o alinhamento com excipiente (*padding*) do último bloco do texto a cifrar, ou seja, acrescentam-se bytes até perfazer a dimensão do bloco. Estes bytes introduzidos são depois retirados após a realização da operação de decifra.

Existem vários padrões para o alinhamento com excipiente, sendo um deles indicado pela norma PKCS#5, que usou nas secções anteriores. Neste exercício pretende-se que demonstre a utilização do *padding* e que o mesmo obedece à norma PKCS#5.

Utilizando o GHex e os programas de cifra e de decifra, que desenvolveu na secção anterior, com um algoritmo de cifra AES e modo de cifra ECB (por exemplo), idealize uma experiência que mostre a utilização do *padding* PKCS#5.

Sugestão: Na cifra use uma transformação com *padding* PKCS#5 e na decifra uma transformação com *padding* NoPadding (o que significa que não usa *padding*).

## 2.2 Modos de Cifra

Nesta secção pretende-se explorar a cifra simétrica por blocos usando os vários modos de cifra estudados. Comece por criar uma cópia do(s) programa(s) criado(s) na secções anteriores.

### 2.2.1 Vetor inicial

Como sabe, os modos de cifra com realimentação (CBC, OFB, CFB e CTR, ou seja, todos os estudados com exceção do ECB) utilizam um vetor inicial. Quando se usa uma instância da classe Cipher, para realizar uma operação de cifra usando um dos modos de cifra atrás indicados, ela gera um vetor inicial aleatório (alternativamente podemos ser nós a fornecê-lo), que pode ser lido e que tem obrigatoriamente de ser fornecido para a realização da correspondente operação de decifra.

Altere o programa de cifra desenvolvido na secção 2.1.4 para que, além dos parâmetros de entrada atuais, receba também o nome do ficheiro para guardar o vetor inicial que foi gerado e utilizado na operação de cifra. Note que o programa deve continuar a permitir o modo de cifra ECB, pelo que apenas deve utilizar o vetor inicial para os modos de cifra acima indicados.

Altere também o programa de decifra que desenvolveu na secção 4 para que, além dos parâmetros de entrada atuais, receba também o nome do ficheiro que contém o vetor inicial a utilizar na operação de decifra. Note que o programa deve continuar a permitir o modo de cifra ECB, pelo que apenas deve utilizar o vetor inicial para os modos de cifra acima indicados.

Sugestão: Considere a utilização da classe IvParameterSpec.

### 2.2.2 Reprodução de padrões

Neste exercício vamos analisar o impacto dos modos cifra ECB e CBC na reprodução de padrões existentes num documento em claro. Para esse efeito, vamos cifrar uma imagem em formato BMP (ficheiro com extensão BMP) e depois visualizá-lo e compará-lo com o ficheiro em claro. No entanto, para se poder visualizar o ficheiro cifrado, o seu cabeçalho não pode estar cifrado (ele é fundamental para que o programa de visualização de imagens interprete o conteúdo do ficheiro como sendo uma imagem), pelo que temos que repor os primeiros 54 bytes a partir do ficheiro original.

Com o seu programa de cifra, que desenvolveu na secção 2.1.2 e alterou na secção 2.2.1, e utilizando o modo de cifra ECB com um algoritmo de cifra simétrica à sua escolha, cifre o ficheiro `pic_original.bmp` que pode descarregar da página da disciplina na plataforma de elearning.

Utilizando um editor de ficheiros em binário ( GHex , por exemplo, em Linux), copie os primeiros 54 bytes do ficheiro em claro e sobreponha-os aos primeiros 54 bytes do ficheiro com a imagem cifrada. Alternativamente, poderá usar a seguinte sequência de comandos Unix, assumindo que `pic_cifrada.bmp` é o ficheiro produzido pela cifra e `pic_cifrada_1.bmp` é o ficheiro onde pretende combinar o cabeçalho original com o conteúdo cifrado:

```
dd if=pic_original.bmp ibs=1 count=54 > pic_cifrada_1.bmp
dd if=pic_cifrada.bmp ibs=1 skip=54 >> pic_cifrada_1.bmp
```

Abra o ficheiro com a imagem em claro e o ficheiro com a imagem cifrada e com os primeiros 54 bytes alterados, utilizando um programa para visualizar imagens e compare o seu conteúdo. O que observa?

Cifre novamente o ficheiro `pic_original.bmp`, agora utilizando o modo de cifra CBC e o mesmo algoritmo de cifra que utilizou atrás. Repita as operações para sobrepor os primeiros 54 bytes do ficheiro com a imagem cifrada, para poder visualizar o conteúdo cifrado como sendo uma imagem. Compare a imagem do ficheiro em claro com a imagem do ficheiro cifrado. O que observa?

Repita a experiência para os mesmos modos de cifra e para vários algoritmos de cifra. O que conclui?

## 2.2.3 Corrupção do criptograma

Neste exercício vamos analisar o impacto que a ocorrência de erros no criptograma induz no correspondente texto em claro, para os modos de cifra ECB, CBC, OFB, CFB e CTR.

Com o seu programa de cifra, que desenvolveu na secção 2.1.2 e alterou na secção 2.2.1, e utilizando o modo de cifra ECB com um algoritmo de cifra à sua escolha (AES, por exemplo), cifre o conteúdo do ficheiro `pic_original.bmp` que descarregou.

Utilizando um editor de ficheiros em binário (GHex, por exemplo), altere um bit a um dos bytes do ficheiro com o conteúdo cifrado, por exemplo ao byte na posição 60H.

Utilizando o seu programa de decifra, que desenvolveu na secção 2.1.3 e alterou na secção 2.2.1, decifre o conteúdo do ficheiro cifrado no qual introduziu o erro, usando o mesmo modo de cifra (ECB) e o mesmo algoritmo que usou para a cifra.

Com um editor de ficheiros em binário (GHex), abra o ficheiro com a imagem original (`pic_original.bmp`) e o ficheiro em claro resultado da decifra no passo anterior. Analise as diferenças entre eles e tire conclusões quanto ao impacto do erro introduzido. Nota: Pode também visualizar o ficheiros, depois de repor os primeiros 54 bytes, como feito anteriormente).

Repita a experiência usando cada um dos restantes modos de cifra e, para cada um deles, retire conclusões quanto ao impacto do erro de um bit no criptograma.

## 2.3 Cifra contínua

### 2.3.1 Reutilização de chaves

Com base no programa para cifrar o conteúdo de um ficheiro, que desenvolveu nas secções 2.1.2 e alterou na secção 2.2.1, crie um outro programa que tenha os mesmos parâmetros de entrada, mais um outro para indicar o nome de um ficheiro com um vetor inicial a usar na operação de cifra. Caso este parâmetro

contenha o nome de um ficheiro, este programa deve usar o vetor inicial nele contido para realizar a operação de cifra (em conjunto com os restantes parâmetros, tal como já fazia). Caso este novo parâmetro esteja vazio (null), o programa deve continuar a cifrar da mesma forma como antes fazia.

Crie um outro programa que faça o XOR do conteúdo de dois ficheiros cujos nomes devem ser fornecidos como parâmetros e guarde o resultado num outro ficheiro cujo nome também deve ser fornecido como parâmetro. O XOR deve ser feito byte a byte. Caso os ficheiros não sejam do mesmo tamanho, a saída deve conter o resultado do XOR de todos os bytes do ficheiro menor com os correspondentes bytes do ficheiro maior, mais os restantes bytes do ficheiro maior.

Crie dois ficheiros com duas mensagens diferentes para cifrar (e.g. mensagem1.txt e mensagem2.txt).

Usando o programa que desenvolveu em 2.1.1, gere uma chave simétrica para um algoritmo à sua escolha (e.g., para o algoritmo AES) e guarde-a num ficheiro.

Usando esta chave, e o programa de cifra que atrás desenvolveu, cifre o conteúdo do primeiro ficheiro, usando uma cifra contínua (i.e., usando uma transformação com um dos algoritmos OFB, CFB e CTR, como por exemplo a transformação AES/OFB/NoPadding) e guarde em ficheiro o criptograma resultante (e.g., no ficheiro mensagem1.cripto) e o vetor inicial usado (e.g., no ficheiro mensagem1.vi).

Usando o mesmo programa de cifra, cifre também o segundo ficheiro, usando a mesma transformação, a mesma chave e o mesmo vetor inicial (que guardou no ficheiro mensagem1.vi) e guarde o criptograma resultante num outro ficheiro (e.g., mensagem2.cripto).

Usando o programa que criou para fazer o XOR de dois ficheiros, faça o XOR dos dois criptogramas que atrás obteve (ficheiros mensagem1.cripto e mensagem2.cripto) e guarde o resultado num novo ficheiro (e.g., mensagens.xor).

Ainda usando o programa que criou para fazer o XOR de dois ficheiros, faça agora o XOR do criptograma da primeira mensagem (mensagem1.cripto) com o resultado do XOR anterior (mensagens.xor) e guarde o resultado num novo ficheiro (e.g., resultado.txt).

Abra o ficheiro que obteve (resultado.txt) e analise o seu conteúdo. O que conclui?

### 2.3.2 Manipulação do criptograma

Crie um ficheiro (e.g., mensagem.txt) com a seguinte mensagem: "Mensagem muito secreta!".

Usando o programa de cifra que desenvolveu na secção anterior, cifre este ficheiro usando uma cifra contínua (e.g., usando a transformação AES/OFB/NoPadding) e guarde o criptograma resultante num ficheiro (e.g., mensagem.cripto).

Usando um editor de ficheiros em binário (e.g., GHex), altere um bit apenas ao criptograma e salve o ficheiro. Decifre o criptograma alterado e compare o resultado com o texto original (mensagem.txt). O que conclui?

Consegue alterar o criptograma de forma a que o resultado da operação de decifra seja a mensagem: "Mensagem pouco secreta!"?

## 2.4 Password Based Encryption (PBE)

Dada a dificuldade dos humanos em lidar com chaves, frequentemente usa-se criptografia com base numa *password* (PBE - Password Based Encryption), em que a chave simétrica a usar é derivada a partir de uma *password* e de um sal (*salt*).

O seguinte exemplo ilustra a geração de uma chave AES com base numa *password*:

```
SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
KeySpec keySpec = new PBEKeySpec(password.toCharArray(), salt, 65536, 256);
SecretKey secretKey = factory.generateSecret(keySpec);
SecretKey secret = new SecretKeySpec(secretKey.getEncoded(), "AES");
```

Desenvolva dois programas que usam AES e *password based encryption*, sendo um para cifrar e o outro para decifrar. Considere que os dados para cifrar e decifrar estão guardados em ficheiros e que os resultados das operações são também guardados em ficheiro. Guarde também em ficheiro todos os dados resultantes da operação de cifra necessários para a operação de decifra.

### 3 Funções de Síntese (*hash*)

Crie um programa para obter a síntese (*one-way, cryptographic hash*) de um texto fornecido sob a forma de um ficheiro. O programa deverá ter os seguintes parâmetros de entrada: (i) o nome do ficheiro com os dados dos quais se pretende calcular a síntese, (ii) o nome da função de síntese a usar (MD5, SHA-1, SHA-256, SHA-384 ou SHA-512), e (iii) um parâmetro opcional que é o nome de um ficheiro onde guardar a síntese obtida. Como este terceiro parâmetro é opcional, o programa deve sempre mostrar a síntese obtida no ecrã (com a forma de número hexadecimal).

Sugestão : use as classes `MessageDigest` e `DatatypeConverter`.

#### 3.1 Efeito de Avalanche ou Difusão

Um requisito importante para as funções de síntese criptográficas é que uma pequena alteração no texto deve produzir uma síntese completamente diferente – efeito de avalanche ou difusão. Vamos verificar esta propriedade.

Utilizando o seu programa para calcular sínteses, que criou atrás, obtenha a síntese do conteúdo de um mesmo ficheiro para uma das funções de síntese (e.g., SHA-256) e guarde-a num ficheiro. Verifique o comprimento da síntese obtida.

Usando um editor de ficheiros em binário (e.g., GHex), altere em apenas um bit o conteúdo do ficheiro com o qual gerou a síntese anterior.

Continuando a usar o seu programa, obtenha a síntese deste novo conteúdo do ficheiro, usando a mesma função de síntese que antes usou, e guarde-a num novo ficheiro.

Compare as duas sínteses obtidas, uma a partir do ficheiro original e a outra a partir do ficheiro com a alteração de um bit ao conteúdo. Quantos bits são diferentes?

Repita a experiência usando outras funções de síntese.

#### 3.2 Análise estatística de sínteses de mensagens semelhantes

Crie um novo programa para calcular sínteses, baseado no anterior, mas cujo resultado deverá ser a distribuição estatística das diferenças entre as sínteses calculadas sobre um conjunto de N mensagens que

diferem em apenas um bit de uma mensagem inicial. O programa deverá receber como parâmetros (i) o nome de um ficheiro com a mensagem (conteúdo) inicial relativa à qual se pretende calcular uma síntese e (ii) o valor de N, isto é o número de mensagens a gerar que diferem da inicial em apenas 1 bit.

Cada uma das N mensagens deverá ser uma cópia da mensagem inicial, na qual foi mudado apenas um bit aleatório. Para escolher esse bit, use um gerador de números aleatórios para obter o seu índice. Após calcular a síntese de cada conteúdo modificado, avalie a diferença entre o mesmo e o da síntese original em termos do número de bits diferentes (distância de Hamming em bits). Comente a distribuição das diferenças de bits entre as sínteses obtidas.

Sugestão : use a classe `Random` para gerar a posição aleatória para o bit a alterar e use a operação XOR para detetar bits alterados em cada byte (os bits com o valor 1 estão nas posições em que houve alteração no hash).

## 4 Código de Autenticação de Mensagens (MAC – *Message Authentication Code*)

Crie um programa para obter o MAC (*Message Authentication Code*) de uma mensagem num ficheiro. O programa deverá ter os seguintes parâmetros de entrada: (i) o nome do ficheiro com a mensagem de que se pretende calcular o MAC, (ii) o nome da função de MAC a usar (`HmacMD5`, `HmacSHA1`, `HmacSHA256`, `HmacSHA384`, `HmacSHA512`), (iii) o nome do ficheiro com a chave a usar, e (iv) o nome do ficheiro onde guardar o código MAC obtido. O programa deve sempre mostrar no ecrã o código MAC obtido, no formato hexadecimal ou base64.

Usando um mesmo algoritmo de MAC e uma mesma chave, compare os códigos MAC que obtém quando calculados usando mensagens que diferem entre si em apenas um bit.

Usando um mesmo algoritmo de MAC, compare os códigos MAC que obtém quando calculados sobre uma mesma mensagem usando diferentes chaves.

Sugestão: Use a classe `javax.crypto/Mac`

## 5 Criptografia de chave assimétrica

### 5.1 Geração de um par de chaves

Desenvolva um pequeno programa para gerar um par de chaves assimétricas para o algoritmo RSA e para um comprimento de chave especificado como parâmetro de entrada (1024, 2048, 3072 ou 4096). O programa deve guardar o par de chaves em dois ficheiros, um para a chave privada e outro para a chave pública, cujos nomes são fornecidos como parâmetro de entrada.

Sugestão: Use as classes `KeyPairGenerator`, `KeyPair`, `PrivateKey` e `PublicKey`.

### 5.2 Cifra com o algoritmo RSA

Desenvolva um pequeno programa para cifrar o conteúdo de um ficheiro utilizando o algoritmo RSA. O nome do ficheiro com o conteúdo a cifrar deve ser passado como parâmetro de entrada, bem como o nome

do ficheiro com a chave pública a utilizar para realizar a cifra e o nome do ficheiro onde guardar o criptograma obtido.

Sugestão: Use as classes `KeyFactory`, `Cipher` e `X509EncodedKeySpec`.

**Nota:** Tenha em atenção o tamanho do ficheiro a cifrar. Utilizando uma chave RSA de 1024 bits (128 bytes) não pode cifrar blocos (de ficheiros) com mais de 117 bytes.

### 5.3 Decifra com o algoritmo RSA

Desenvolva um pequeno programa para decifrar o conteúdo de um ficheiro, cujo nome deve ser fornecido como parâmetro de entrada, utilizando o algoritmo RSA. Deve também fornecer como parâmetro de entrada o nome do ficheiro com a chave privada a utilizar para a operação de decifra e o nome do ficheiro onde se deve salvar a mensagem decifrada.

Sugestão: Use as classes `KeyFactory`, `PrivateKey`, `Cipher` e `PKCS8EncodedKeySpec`.

### 5.4 Cifras sucessivas do mesmo texto produzem o quê?

Utilizando o programa de cifra assimétrica que desenvolveu em 5.2, cifre duas vezes um mesmo texto, utilizando a mesma chave, e guarde os criptogramas gerados em dois ficheiros diferentes. Num editor de ficheiros em binário (e.g., o GHex), abra os dois ficheiros com os criptogramas gerados e compare-os. O que conclui?

Use agora o programa de decifra que desenvolveu em 5.3 e decifre os dois criptogramas gerados, para dois novos ficheiros diferentes. Compare os ficheiros com o resultado da decifra. O que conclui? Consegue explicar o porquê do que observou?

### 5.5 Como cifrar um ficheiro grande? Cifra híbrida

Como já foi referido, utilizando RSA e uma chave de 1024 bits, não consegue cifrar textos com uma dimensão superior a 117 bytes. Imagine que possui a chave pública de uma determinada pessoa e que pretende enviar-lhe um ficheiro muito grande, por exemplo 100 MBytes, de modo que apenas essa pessoa o possa decifrar. Não tem forma de contactar essa pessoa de forma a previamente combinar uma chave simétrica partilhada, pelo que vai ter de obrigatoriamente usar a chave pública RSA dessa pessoa, embora possa também usar outros algoritmos de cifra. Dividir o ficheiro grande em blocos de 117 bytes e cifrar cada um deles com a chave pública está fora de questão porque ia ser uma operação demorada. Consegue indicar uma forma de enviar o ficheiro, cifrado de forma eficiente e de modo que apenas o dono da chave pública o possa decifrar?

## 6 Bibliografia

Java Cryptography Architecture (JCA) Reference Guide,

<https://docs.oracle.com/javase/8/docs/technotes/guides/security/crypto/CryptoSpec.html>

The Java Tutorials: Security Features in Java SE, <http://docs.oracle.com/javase/tutorial/security/index.html>

